Overview of Backpropagation

Subject: Backpropagation

$$\frac{\partial L}{\partial W} = \delta \cdot a^T$$

$$\delta^{(l)} = (W^{(l+1)T}\delta^{(l+1)}) \odot \sigma'(z^{(l)})$$

1.

∂ E ∂ w i j = ∂ E ∂ o j ∂ o j ∂ net j ∂ net j ∂ w i j = ∂ E ∂ o j ∂ o j ∂ net j o i {\displaystyle {\frac {\partial E}{\partial w_{ij}}}={\frac {\partial E}{\partial o_{j}}}{\frac {\partial o_{j}}{\partial {\text{net}}_{j}}}{\frac {\partial {\text{net}}_{j}}{\partial w_{ij}}}={\frac {\partial E}{\partial o_{j}}}{\frac {\partial o_{j}}{\partial {\text{net}}_{j}}}o_{i}}

2.

Note that δ l {\displaystyle \delta ^{l}} is a vector, of length equal to the number of nodes in level l {\displaystyle l} ; each component is interpreted as the "cost attributable to (the value of) that node". The gradient of the weights in layer l {\displaystyle l} is then:

3.

The gradient ∇ {\displaystyle \nabla } is the transpose of the derivative of the output in terms of the input, so the matrices are transposed and the order of multiplication is reversed, but the entries are the same:

4.

Gradient descent with backpropagation is not guaranteed to find the global minimum of the error function, but only a local minimum; also, it has trouble crossing plateaus in the error function landscape. This issue, caused by the non-convexity of error functions in neural networks, was long thought to be a major drawback, but Yann LeCun et al. argue that in many practical problems, it is not. Backpropagation learning does not require normalization of input vectors; however, normalization could improve performance. Backpropagation requires the derivatives of activation functions to be known at network design time.

5.

C (y , f L (W L f L − 1 (W L − 1 ■ f 2 (W 2 f 1 (W 1 x)) ■))) {\displaystyle C(y,f^{L}(W^{L}f^{L-1}(W^{L-1}\cdots f^{2}(W^{2}f^{1}(W^{1}x))\cdots)))}

6.

y {\displaystyle y} : target output For classification, output will be a vector of class probabilities (e.g., (0.1 , 0.7 , 0.2) {\displaystyle (0.1,0.7,0.2)} , and target output is a specific class, encoded by the one-hot/dummy variable (e.g., (0 , 1 , 0) {\displaystyle (0,1,0)}).

7.

In machine learning, backpropagation is a gradient computation method commonly used for training a neural network in computing parameter updates. It is an efficient application of the chain rule to neural networks. Backpropagation efficiently computes the gradient of the loss with respect to the network weights for a single input–output example. It does this by propagating derivatives backward, one layer at a time, from the output layer to the input layer, thereby avoiding redundant chain-rule calculations. Strictly speaking, the term backpropagation refers only to an algorithm for efficiently computing the gradient, not how the gradient is used, but the term is often used loosely to refer to the entire learning algorithm. This includes changing model parameters in the negative direction of the gradient, such as by stochastic gradient descent, or as an intermediate step in a more complicated optimizer, such as Adaptive Moment Estimation. Backpropagation had multiple discoveries and partial discoveries, with a tangled history and terminology (see § History). Some other names for the technique include "reverse mode of automatic differentiation" or "reverse accumulation".

8.

The loss function is a function that maps values of one or more variables onto a real number intuitively representing some "cost" associated with those values. For backpropagation, the loss function calculates the difference between the network output and its expected output, after a training example has propagated through the network.

9.

where d a L d z L {\displaystyle {\frac {da^{L}}{dz^{L}}}} is a diagonal matrix. These terms are: the derivative of the loss function; the derivatives of the activation functions; and the matrices of weights:

10.

Motivation The goal of any supervised learning algorithm is to find a function that best maps a set of inputs to their correct output. The motivation for backpropagation is to train a multi-layered neural network such that it can learn the appropriate internal representations to allow it to learn any arbitrary mapping of input to output.

11.

Consider the network on a single training case: (1 , 1 , 0) {\displaystyle (1,1,0)} . Thus, the input x 1 {\displaystyle x_{1}} and x 2 {\displaystyle x_{2}} are 1 and 1 respectively and the correct output, t is 0. Now if the relation is plotted between the network's output y on the horizontal axis and the error E on the vertical axis, the result is a parabola. The minimum of the parabola corresponds to the output y which minimizes the error E. For a single training case, the minimum also touches the horizontal axis, which means the error will be zero and the network can produce an output y that exactly matches the target output t. Therefore, the problem of mapping inputs to outputs can be reduced to an optimization problem of finding a function that will produce the minimal error. However, the output of a neuron depends on the weighted sum of all its inputs:

12.

d C d a L · d a L d z L · d z L d a L − 1 · d a L − 1 d z L − 1 · d z L − 1 d a L − 2 · … · d a 1 d z 1 · ∂ z 1 ∂ x , {\displaystyle {\frac {dC}{da^{L}}}\cdot {\frac {da^{L}}{dz^{L}}}\cdot {\frac {dz^{L}}{da^{L-1}}}\cdot {\frac {da^{L-1}}{dz^{L-1}}}\cdot {\frac {dz^{L-1}}{da^{L-2}}}\cdot \ldots \cdot {\frac {da^{1}}{dz^{1}}}\cdot {\frac {\partial z^{1}}{\partial x}},}

13.

δ l := (f l) ′ ■ (W l + 1) T · (f l + 1) ′ ■ ■ ■ (W L − 1) T · (f L − 1) ′ ■ (W L) T · (f L) ′ ■ ∇ a L C . {\displaystyle \delta ^{l}:=(f^{l})'\circ (W^{l+1})^{T}\cdot (f^{l+1})'\circ \cdots \circ (W^{L-1})^{T}\cdot (f^{L-1})'\circ (W^{L})^{T}\cdot (f^{L})'\circ \nabla _{a^{L}}C.}

14.

Backpropagation then consists essentially of evaluating this expression from right to left (equivalently, multiplying the previous expression for the derivative from left to right), computing the gradient at each layer on the way; there is an added step, because the gradient of the weights is not just a subexpression: there's an extra multiplication. Introducing the auxiliary quantity δ l {\displaystyle \delta ^{l}} for the partial products (multiplying from right to left), interpreted as the "error at level l {\displaystyle l} " and defined as the gradient of the input values at level l {\displaystyle l} :